

Lab Experiment 5 : Linear Regression through Gradient Descent

Aim

To implement Linear Regression using the Gradient Descent optimization algorithm and evaluate its performance on a real world dataset.

Task 1: Download and Load the Dataset

We will download the Student Performance dataset directly from the UCI repository using pandas.

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import warnings
# Suppress notebook warning noise
warnings.filterwarnings("ignore")

# Set random seed so results are reproducible across runs
np.random.seed(42)

# Load the dataset (student-mat.csv)
df = pd.read_csv("student-mat.csv", sep=";")

print("Dataset loaded successfully!")
print(f"Shape: {df.shape}")
print(f"\nFirst 5 rows:\n{df.head()}")
print(f"\nColumn names:\n{df.columns.tolist()}")
```

Dataset loaded successfully!

Shape: (395, 33)

First 5 rows:

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob
0	GP	F	18	U	GT3	A	4	4	at_home	teacher
1	GP	F	17	U	GT3	T	1	1	at_home	other
2	GP	F	15	U	LE3	T	1	1	at_home	other
3	GP	F	15	U	GT3	T	4	2	health	services
4	GP	F	16	U	GT3	T	3	3	other	other

	famrel	freetime	goout	Dalc	Walc	health	absences	G1	G2	G3
0	4	3	4	1	1	3	6	5	6	6
1	5	3	3	1	1	3	4	5	5	6
2	4	3	2	2	3	3	10	7	8	10
3	3	2	2	1	1	5	2	15	14	15
4	4	3	2	1	2	5	4	6	10	10

[5 rows x 33 columns]

Column names:

['school', 'sex', 'age', 'address', 'famsize', 'Pstatus', 'Medu', 'Fedu', 'Mjob', 'Fjob', 'reason', 'guardian', 'traveltime', 'studytime', 'failures', 'schoolsup', 'famsup', 'paid', 'activities', 'nursery', 'higher', 'internet', 'romantic', 'famrel', 'freetime', 'goout', 'Dalc', 'Walc', 'health', 'absences', 'G1', 'G2', 'G3']

Task 2: Data Preprocessing

In this section, we will:

- Check for missing values
- Explore the dataset structure
- Encode categorical variables
- Scale numerical features

```
In [2]: # Check for missing values
print("Missing values per column:")
print(df.isnull().sum())
print(f"\nTotal missing values: {df.isnull().sum().sum()}")

# Dataset info
print("\nDataset Info:")
print(df.dtypes)

# Summary statistics
```

```
print("\nSummary Statistics:")  
print(df.describe())
```

```
Missing values per column:
school      0
sex         0
age         0
address     0
famsize     0
Pstatus     0
Medu        0
Fedu        0
Mjob        0
Fjob        0
reason      0
guardian    0
traveltime  0
studytime   0
failures    0
schoolsup   0
famsup      0
paid        0
activities  0
nursery     0
higher      0
internet    0
romantic    0
famrel      0
freetime    0
goout       0
Dalc        0
Walc        0
health      0
absences    0
G1          0
G2          0
G3          0
dtype: int64
```

Total missing values: 0

```
Dataset Info:
school      str
sex         str
age         int64
address     str
famsize     str
Pstatus     str
Medu        int64
Fedu        int64
Mjob        str
Fjob        str
reason      str
guardian    str
traveltime  int64
studytime   int64
failures    int64
schoolsup   str
famsup      str
```

```

paid          str
activities    str
nursery       str
higher        str
internet      str
romantic      str
famrel        int64
freetime      int64
goout         int64
Dalc          int64
Walc          int64
health        int64
absences      int64
G1            int64
G2            int64
G3            int64
dtype: object

```

Summary Statistics:

	age	Medu	Fedu	traveltime	studytime	failure
s \ count	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000
mean	16.696203	2.749367	2.521519	1.448101	2.035443	0.33417
std	1.276043	1.094735	1.088201	0.697505	0.839240	0.74365
min	15.000000	0.000000	0.000000	1.000000	1.000000	0.000000
25%	16.000000	2.000000	2.000000	1.000000	1.000000	0.000000
50%	17.000000	3.000000	2.000000	1.000000	2.000000	0.000000
75%	18.000000	4.000000	3.000000	2.000000	2.000000	0.000000
max	22.000000	4.000000	4.000000	4.000000	4.000000	3.000000

	famrel	freetime	goout	Dalc	Walc	healt
h \ count	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000
mean	3.944304	3.235443	3.108861	1.481013	2.291139	3.55443
std	0.896659	0.998862	1.113278	0.890741	1.287897	1.39030
min	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000
25%	4.000000	3.000000	2.000000	1.000000	1.000000	3.000000
50%	4.000000	3.000000	3.000000	1.000000	2.000000	4.000000
75%	5.000000	4.000000	4.000000	2.000000	3.000000	5.000000
max	5.000000	5.000000	5.000000	5.000000	5.000000	5.000000

	absences	G1	G2	G3
count	395.000000	395.000000	395.000000	395.000000
mean	5.708861	10.908861	10.713924	10.415190
std	8.003096	3.319195	3.761505	4.581443
min	0.000000	3.000000	0.000000	0.000000
25%	0.000000	8.000000	9.000000	8.000000
50%	4.000000	11.000000	11.000000	11.000000
75%	8.000000	13.000000	13.000000	14.000000
max	75.000000	19.000000	19.000000	20.000000

Handling Categorical Variables

We will use OneHotEncoder for categorical features and StandardScaler for numerical features.

```
In [3]: # Identify categorical and numerical columns
categorical_cols = df.select_dtypes(include=["object"]).columns.tolist()
numerical_cols = df.select_dtypes(include=["int64", "float64"]).columns.tolist()

print(f"Categorical columns ({len(categorical_cols)}): {categorical_cols}")
print(f"\nNumerical columns ({len(numerical_cols)}): {numerical_cols}")
```

Categorical columns (17): ['school', 'sex', 'address', 'famsize', 'Pstatus', 'Mjob', 'Fjob', 'reason', 'guardian', 'schoolsup', 'famsup', 'paid', 'activities', 'nursery', 'higher', 'internet', 'romantic']

Numerical columns (16): ['age', 'Medu', 'Fedu', 'traveltime', 'studytime', 'failures', 'famrel', 'freetime', 'goout', 'Dalc', 'Walc', 'health', 'absences', 'G1', 'G2', 'G3']

```
In [4]: # Define target variable (G3 - final grade)
target = "G3"

# Remove G1 and G2 to avoid data leakage (G3 is our target)
features = [col for col in numerical_cols if col not in ["G1", "G2", "G3"]]

print(f"Features to use ({len(features)}): {features}")
print(f"Target: {target}")
```

Features to use (30): ['age', 'Medu', 'Fedu', 'traveltime', 'studytime', 'failures', 'famrel', 'freetime', 'goout', 'Dalc', 'Walc', 'health', 'absences', 'school', 'sex', 'address', 'famsize', 'Pstatus', 'Mjob', 'Fjob', 'reason', 'guardian', 'schoolsup', 'famsup', 'paid', 'activities', 'nursery', 'higher', 'internet', 'romantic']
Target: G3

Correlation Heatmap

Pairwise correlations between numerical features.

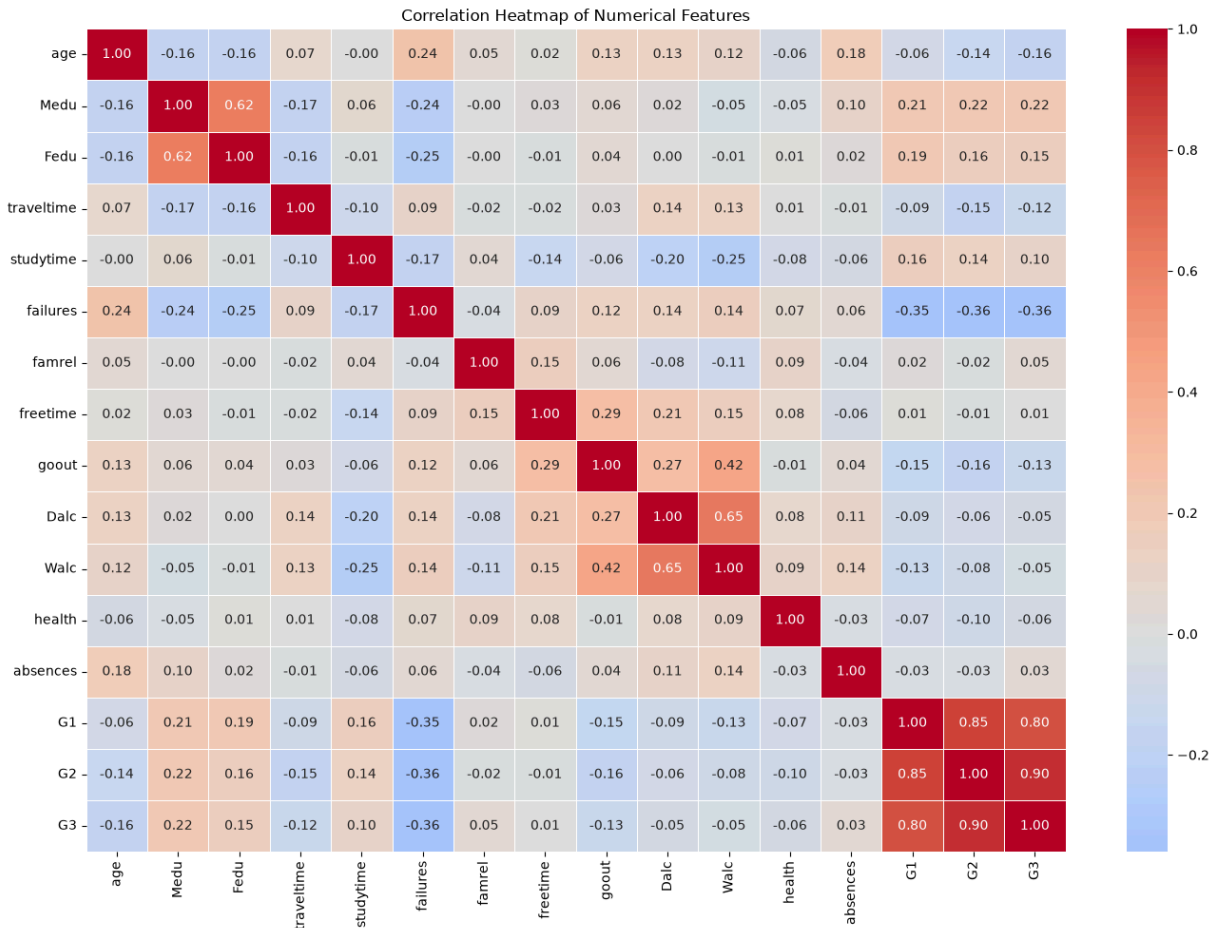
```
In [5]: # Correlation heatmap of numerical features
plt.figure(figsize=(14, 10))
corr_matrix = df[numerical_cols].corr()
```

```

sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap="coolwarm", center=0, 1
plt.title("Correlation Heatmap of Numerical Features")
plt.tight_layout()
plt.show()

# Identify which features are most predictive of G3
target_corr = corr_matrix["G3"].drop("G3").sort_values(ascending=False)
print("\nCorrelation with G3 (target):")
print(target_corr)

```



Correlation with G3 (target):

```

G2          0.904868
G1          0.801468
Medu        0.217147
Fedu        0.152457
studytime   0.097820
famrel       0.051363
absences    0.034247
freetime    0.011307
Walc        -0.051939
Dalc        -0.054660
health      -0.061335
traveltime  -0.117142
goout       -0.132791
age         -0.161579
failures    -0.360415
Name: G3, dtype: float64

```

Outlier Detection

IQR-based outlier detection on G3 and key features.

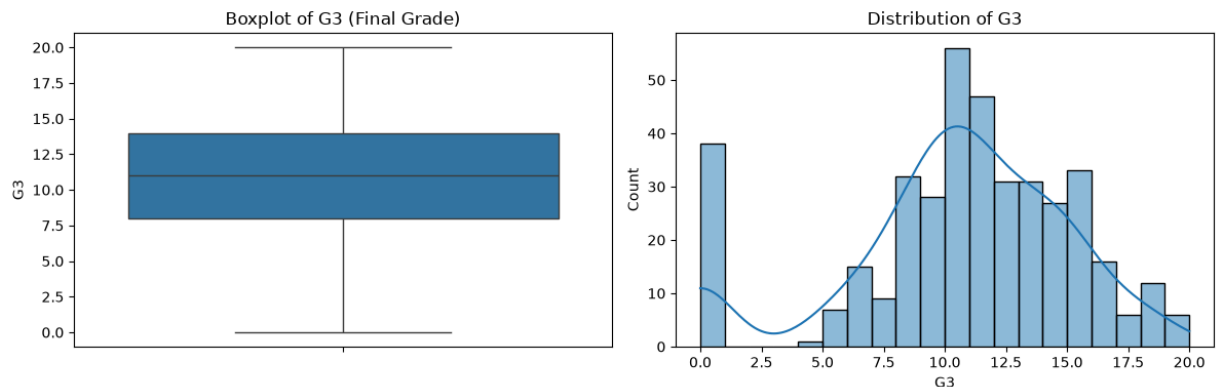
```
In [6]: # IQR: values beyond 1.5x the interquartile range are outliers
def detect_outliers_iqr(series, factor=1.5):
    q1 = series.quantile(0.25)
    q3 = series.quantile(0.75)
    iqr = q3 - q1
    lower = q1 - factor * iqr
    upper = q3 + factor * iqr
    return (series < lower) | (series > upper)

# Check outliers in target and top numerical features
check_cols = ["G3", "studytime", "failures", "absences"]
outlier_summary = {}
for col in check_cols:
    mask = detect_outliers_iqr(df[col])
    outlier_summary[col] = mask.sum()
    print(f"{col}: {mask.sum()} outliers ({mask.sum()/len(df)*100:.1f}%)")

# Visualize outliers in G3
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
sns.boxplot(y=df["G3"], ax=axes[0])
axes[0].set_title("Boxplot of G3 (Final Grade)")
sns.histplot(df["G3"], bins=20, kde=True, ax=axes[1])
axes[1].set_title("Distribution of G3")
plt.tight_layout()
plt.show()

# Clean data improves gradient descent convergence
outlier_mask = detect_outliers_iqr(df["G3"])
df_clean = df[~outlier_mask].copy()
print(f"\nOriginal dataset: {len(df)} rows")
print(f"After outlier removal: {len(df_clean)} rows ({outlier_mask.sum()} re
```

G3: 0 outliers (0.0%)
 studytime: 27 outliers (6.8%)
 failures: 83 outliers (21.0%)
 absences: 15 outliers (3.8%)



Original dataset: 395 rows
 After outlier removal: 395 rows (0 removed)

Task 3: Split Dataset into Training and Testing Sets

We will use 80% for training and 20% for testing.

```
In [7]: X = df_clean[features]
        y = df_clean[target]

        # 80/20 split to evaluate how well model generalizes
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

        print(f"Training set shape: {X_train.shape}")
        print(f"Testing set shape: {X_test.shape}")
```

Training set shape: (316, 30)

Testing set shape: (79, 30)

Task 4: Implement Linear Regression using Gradient Descent

We will implement Linear Regression from scratch using the Gradient Descent optimization algorithm.

```
In [8]: class LinearRegressionGD:
        """
        Linear Regression using Gradient Descent
        """
        def __init__(self, learning_rate=0.01, n_iterations=1000):
            self.learning_rate = learning_rate
            self.n_iterations = n_iterations
            self.weights = None
            self.bias = None
            self.loss_history = []

        def _compute_loss(self, X, y):
            """MSE/2 loss - the 1/2 cancels with the exponent derivative"""
            n = len(y)
            predictions = X.dot(self.weights) + self.bias
            loss = (1 / (2 * n)) * np.sum((predictions - y) ** 2)
            return loss

        def fit(self, X, y):
            n_samples, n_features = X.shape
            self.weights = np.zeros(n_features)
            self.bias = 0
            self.loss_history = []

            for i in range(self.n_iterations):
                predictions = X.dot(self.weights) + self.bias

                # Compute gradients
                dw = (1 / n_samples) * X.T.dot(predictions - y)
                db = (1 / n_samples) * np.sum(predictions - y)
```

```

        # Update parameters
        self.weights -= self.learning_rate * dw
        self.bias -= self.learning_rate * db

        # Record loss
        loss = self._compute_loss(X, y)
        self.loss_history.append(loss)

    return self

    def predict(self, X):
        return X.dot(self.weights) + self.bias

print("LinearRegressionGD class defined successfully!")

```

LinearRegressionGD class defined successfully!

```

In [9]: # Create preprocessing pipeline
numeric_features = [col for col in features if col in numerical_cols]
categorical_features = [col for col in features if col in categorical_cols]

# Scale to zero mean/unit variance - GD converges faster on normalized features
numeric_transformer = Pipeline(steps=[
    ("scaler", StandardScaler())
])

# OneHot: categories become binary columns, drop_first avoids multicollinearity
categorical_transformer = Pipeline(steps=[
    ("onehot", OneHotEncoder(drop="first", sparse_output=False, handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features)
    ]
)

# Fit preprocessor on training data and transform both
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

print(f"Processed training shape: {X_train_processed.shape}")
print(f"Processed testing shape: {X_test_processed.shape}")

```

Processed training shape: (316, 39)

Processed testing shape: (79, 39)

Task 5: Experiment with Different Learning Rates

Let us train the model with different learning rates and observe their effect on convergence.

```
In [10]: # LR too small = slow convergence, too large = diverge
learning_rates = [0.001, 0.01, 0.1, 0.5]
models = {}

for lr in learning_rates:
    model = LinearRegressionGD(learning_rate=lr, n_iterations=1000)
    model.fit(X_train_processed, y_train.values)
    models[lr] = model
    print(f"Learning Rate {lr}: Final Loss = {model.loss_history[-1]:.4f}")
```

Learning Rate 0.001: Final Loss = 9.1965

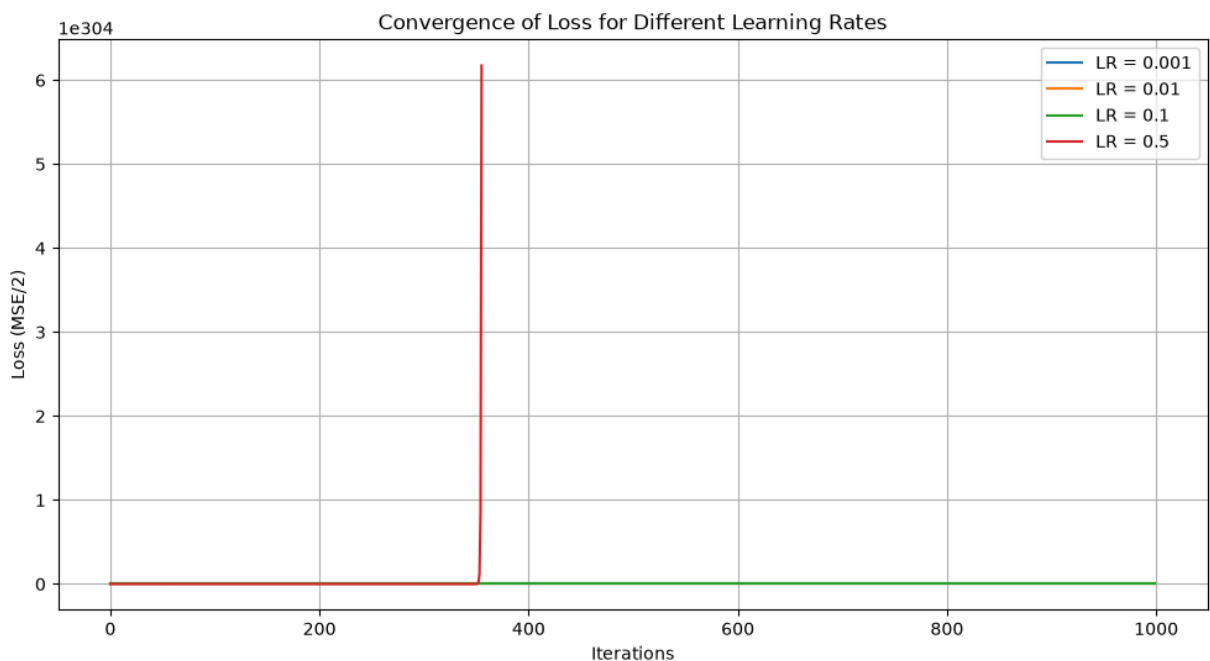
Learning Rate 0.01: Final Loss = 7.8879

Learning Rate 0.1: Final Loss = 7.5020

Learning Rate 0.5: Final Loss = nan

```
In [11]: # Plot loss convergence for different learning rates
plt.figure(figsize=(12, 6))
for lr, model in models.items():
    plt.plot(model.loss_history, label=f"LR = {lr}")

plt.xlabel("Iterations")
plt.ylabel("Loss (MSE/2)")
plt.title("Convergence of Loss for Different Learning Rates")
plt.legend()
plt.grid(True)
plt.show()
```



Task 5b: Gradient Descent Variants

Batch vs Stochastic vs Mini-batch GD comparison.

```
In [12]: # SGD: updates weights after every single sample – noisier but faster per st
class StochasticLinearRegressionGD:
```

```

def __init__(self, learning_rate=0.01, n_epochs=50):
    self.learning_rate = learning_rate
    self.n_epochs = n_epochs
    self.weights = None
    self.bias = None
    self.loss_history = []

def fit(self, X, y):
    n_samples, n_features = X.shape
    self.weights = np.zeros(n_features)
    self.bias = 0
    self.loss_history = []

    for epoch in range(self.n_epochs):
        indices = np.random.permutation(n_samples)
        for i in indices:
            xi = X[i:i+1]
            yi = y[i:i+1]
            pred = xi.dot(self.weights) + self.bias
            error = pred - yi
            self.weights -= self.learning_rate * xi.T.dot(error).flatten()
            self.bias -= self.learning_rate * error.sum()
        # Record loss per epoch
        predictions = X.dot(self.weights) + self.bias
        loss = (1 / (2 * n_samples)) * np.sum((predictions - y) ** 2)
        self.loss_history.append(loss)
    return self

def predict(self, X):
    return X.dot(self.weights) + self.bias

# Mini-batch: compromise between full batch and SGD - stable + fast
class MiniBatchLinearRegressionGD:
    def __init__(self, learning_rate=0.01, n_epochs=50, batch_size=32):
        self.learning_rate = learning_rate
        self.n_epochs = n_epochs
        self.batch_size = batch_size
        self.weights = None
        self.bias = None
        self.loss_history = []

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.bias = 0
        self.loss_history = []

        for epoch in range(self.n_epochs):
            indices = np.random.permutation(n_samples)
            for start in range(0, n_samples, self.batch_size):
                end = min(start + self.batch_size, n_samples)
                batch_idx = indices[start:end]
                xi = X[batch_idx]
                yi = y[batch_idx]
                pred = xi.dot(self.weights) + self.bias

```

```

        error = pred - yi
        dw = (1 / len(batch_idx)) * xi.T.dot(error)
        db = (1 / len(batch_idx)) * error.sum()
        self.weights -= self.learning_rate * dw
        self.bias -= self.learning_rate * db
        predictions = X.dot(self.weights) + self.bias
        loss = (1 / (2 * n_samples)) * np.sum((predictions - y) ** 2)
        self.loss_history.append(loss)
    return self

def predict(self, X):
    return X.dot(self.weights) + self.bias

print("SGD and Mini-batch GD classes defined successfully!")

```

SGD and Mini-batch GD classes defined successfully!

```

In [13]: # Select best LR from Task 5
best_lr = min(models.keys(), key=lambda lr: models[lr].loss_history[-1])

# Train all three variants
batch_gd = LinearRegressionGD(learning_rate=best_lr, n_iterations=200)
batch_gd.fit(X_train_processed, y_train.values)

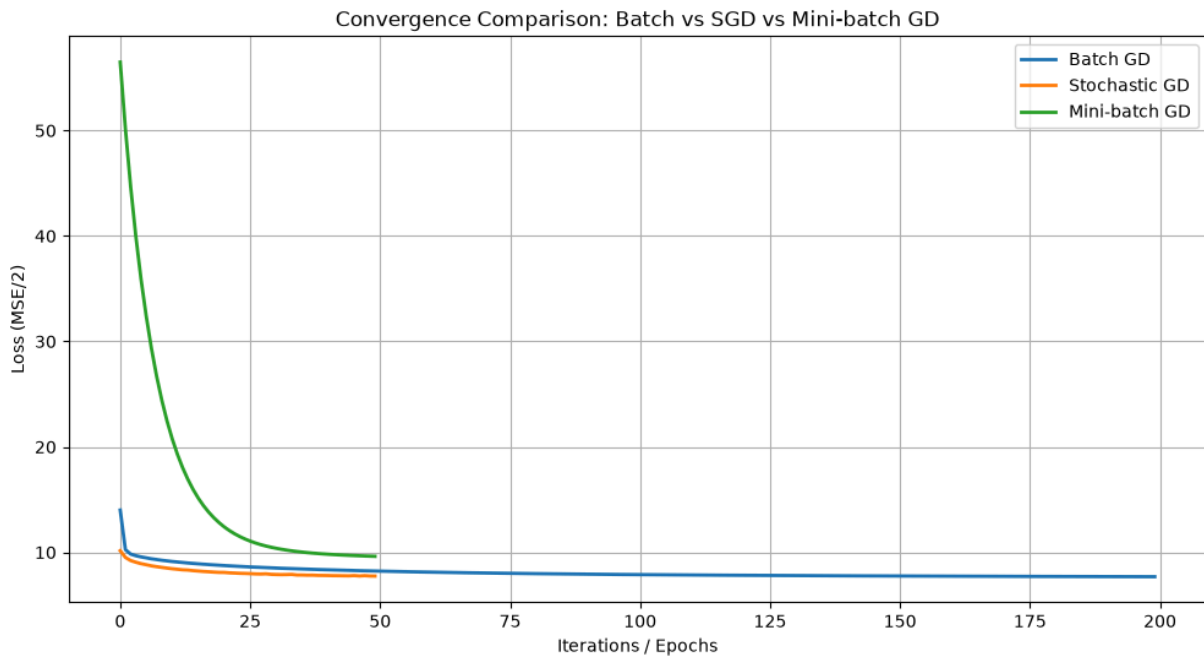
# SGD/Mini-batch need lower LR since they see less data per update
sgd = StochasticLinearRegressionGD(learning_rate=0.001, n_epochs=50)
sgd.fit(X_train_processed, y_train.values)

minibatch_gd = MiniBatchLinearRegressionGD(learning_rate=0.001, n_epochs=50,
minibatch_gd.fit(X_train_processed, y_train.values)

# Plot convergence comparison
plt.figure(figsize=(12, 6))
plt.plot(batch_gd.loss_history, label="Batch GD", linewidth=2)
plt.plot(sgd.loss_history, label="Stochastic GD", linewidth=2)
plt.plot(minibatch_gd.loss_history, label="Mini-batch GD", linewidth=2)
plt.xlabel("Iterations / Epochs")
plt.ylabel("Loss (MSE/2)")
plt.title("Convergence Comparison: Batch vs SGD vs Mini-batch GD")
plt.legend()
plt.grid(True)
plt.show()

# Compare final metrics on test set
print("\nTest Set Performance Comparison:")
print("-" * 45)
for name, model in [("Batch GD", batch_gd), ("SGD", sgd), ("Mini-batch GD",
y_pred = model.predict(X_test_processed)
r2 = r2_score(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
print(f"{name:15s} | R2: {r2:.4f} | RMSE: {rmse:.4f}")

```



Test Set Performance Comparison:

Batch GD	R2: 0.1295 RMSE: 4.2248
SGD	R2: 0.1209 RMSE: 4.2457
Mini-batch GD	R2: -0.0218 RMSE: 4.5773

Task 5c: 3D Loss Surface Visualization

Visualize the GD path on a 3D loss surface using two most important features.

```
In [14]: # Pick two most correlated features for 2D visualization
top2 = target_corr.abs().sort_values(ascending=False).index[:2]
print(f"Using features: {top2[0]} and {top2[1]}")

# Train 2-feature GD model and record weight path
X_2d = X_train_processed[:, [list(numerical_cols).index(top2[0]), list(numerical_cols).index(top2[1])]]
y_2d = y_train.values

path_model = LinearRegressionGD(learning_rate=best_lr, n_iterations=300)
# Override fit to record weight path
n_samples, n_features = X_2d.shape
path_model.weights = np.zeros(n_features)
path_model.bias = 0
path_model.loss_history = []
weight_path = []

for i in range(300):
    predictions = X_2d.dot(path_model.weights) + path_model.bias
    dw = (1 / n_samples) * X_2d.T.dot(predictions - y_2d)
    db = (1 / n_samples) * np.sum(predictions - y_2d)
    path_model.weights -= path_model.learning_rate * dw
    path_model.bias -= path_model.learning_rate * db
    weight_path.append(path_model.weights.copy())
    loss = (1 / (2 * n_samples)) * np.sum((predictions - y_2d) ** 2)
```

```

    path_model.loss_history.append(loss)

weight_path = np.array(weight_path)

# Create meshgrid of weight values
w1_range = np.linspace(weight_path[:, 0].min() - 0.5, weight_path[:, 0].max(
w2_range = np.linspace(weight_path[:, 1].min() - 0.5, weight_path[:, 1].max(
W1, W2 = np.meshgrid(w1_range, w2_range)

# Compute loss surface
Z = np.zeros_like(W1)
for i in range(W1.shape[0]):
    for j in range(W1.shape[1]):
        preds = X_2d.dot([W1[i, j], W2[i, j]]) + path_model.bias
        Z[i, j] = (1 / (2 * n_samples)) * np.sum((preds - y_2d) ** 2)

# 3D surface plot with GD path
fig = plt.figure(figsize=(16, 6))

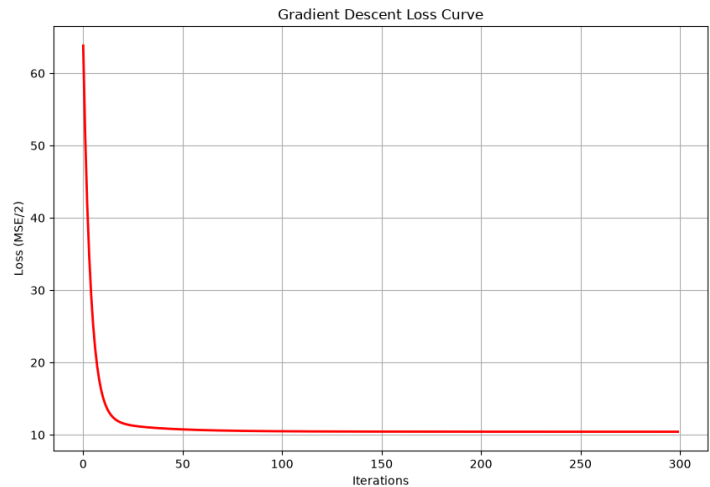
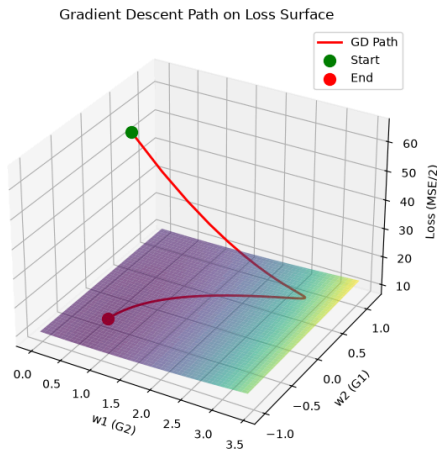
# Left: 3D loss surface with path
ax1 = fig.add_subplot(121, projection="3d")
ax1.plot_surface(W1, W2, Z, cmap="viridis", alpha=0.6)
ax1.plot(weight_path[:, 0], weight_path[:, 1], path_model.loss_history,
        color="red", linewidth=2, label="GD Path")
ax1.scatter(weight_path[0, 0], weight_path[0, 1], path_model.loss_history[0],
        color="green", s=100, zorder=5, label="Start")
ax1.scatter(weight_path[-1, 0], weight_path[-1, 1], path_model.loss_history[-1],
        color="red", s=100, zorder=5, label="End")
ax1.set_xlabel(f"w1 ({top2[0]})")
ax1.set_ylabel(f"w2 ({top2[1]})")
ax1.set_zlabel("Loss (MSE/2)")
ax1.set_title("Gradient Descent Path on Loss Surface")
ax1.legend()

# Right: GD loss curve
ax2 = fig.add_subplot(122)
ax2.plot(path_model.loss_history, color="red", linewidth=2)
ax2.set_xlabel("Iterations")
ax2.set_ylabel("Loss (MSE/2)")
ax2.set_title("Gradient Descent Loss Curve")
ax2.grid(True)

plt.tight_layout()
plt.show()

```

Using features: G2 and G1



Task 6: Model Evaluation

We will evaluate the trained model using standard regression metrics:

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- R^2 Score

```
In [15]: # Select the best model (lowest final loss)
best_lr = min(models.keys(), key=lambda lr: models[lr].loss_history[-1])
best_model = models[best_lr]
print(f"Best learning rate: {best_lr}")

# Make predictions
y_train_pred = best_model.predict(X_train_processed)
y_test_pred = best_model.predict(X_test_processed)

# Calculate metrics
def evaluate_model(y_true, y_pred, dataset_name):
    mae = mean_absolute_error(y_true, y_pred)
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_true, y_pred)
    print(f"\n{dataset_name} Metrics:")
    print(f"  MAE : {mae:.4f}")
    print(f"  MSE : {mse:.4f}")
    print(f"  RMSE : {rmse:.4f}")
    print(f"  R² : {r2:.4f}")
    return mae, mse, rmse, r2

train_metrics = evaluate_model(y_train.values, y_train_pred, "Training Set")
test_metrics = evaluate_model(y_test.values, y_test_pred, "Testing Set")
```

Best learning rate: 0.1

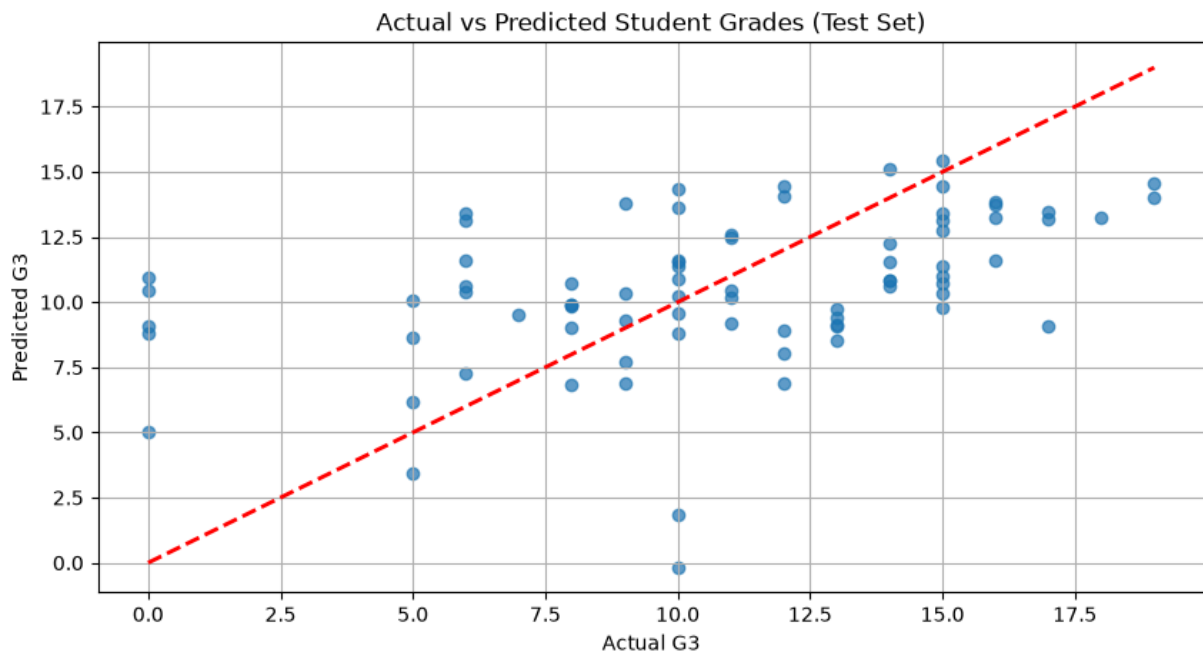
Training Set Metrics:

MAE : 2.9587
MSE : 15.0040
RMSE : 3.8735
 R^2 : 0.2857

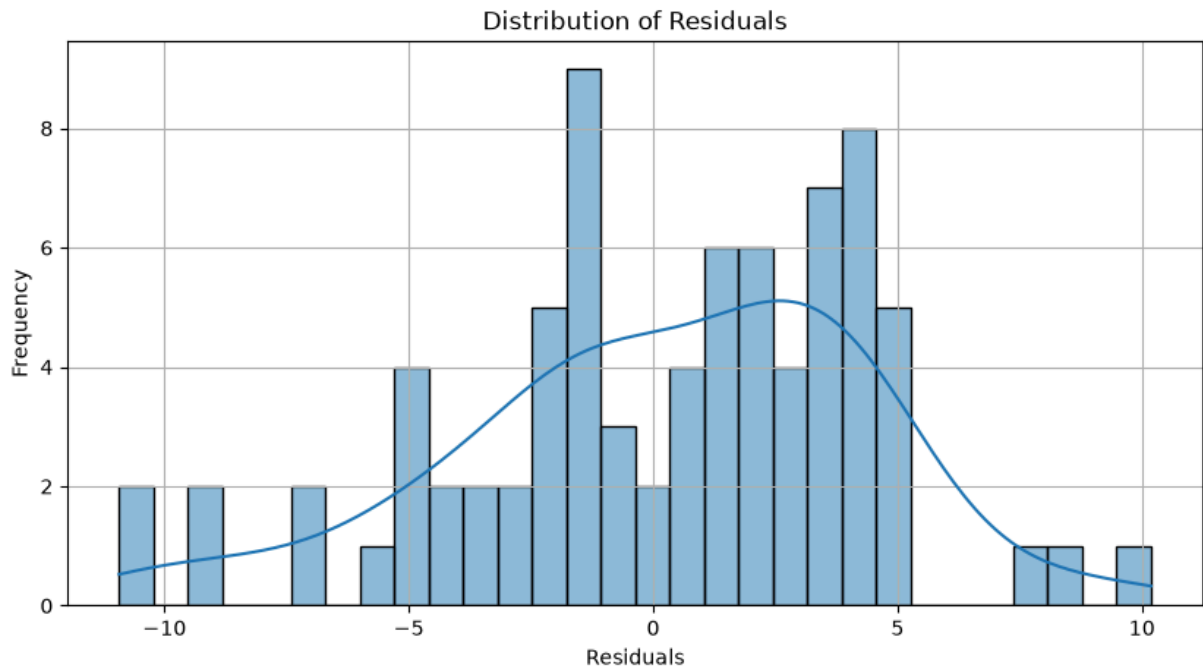
Testing Set Metrics:

MAE : 3.4060
MSE : 17.6304
RMSE : 4.1989
 R^2 : 0.1402

```
In [16]: # Plot actual vs predicted values
plt.figure(figsize=(10, 5))
plt.scatter(y_test, y_test_pred, alpha=0.7)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], "r--",
plt.xlabel("Actual G3")
plt.ylabel("Predicted G3")
plt.title("Actual vs Predicted Student Grades (Test Set)")
plt.grid(True)
plt.show()
```



```
In [17]: # Linear regression assumes residuals are normally distributed
residuals = y_test.values - y_test_pred
plt.figure(figsize=(10, 5))
sns.histplot(residuals, kde=True, bins=30)
plt.xlabel("Residuals")
plt.ylabel("Frequency")
plt.title("Distribution of Residuals")
plt.grid(True)
plt.show()
```



Task 7: Interpretation of Results

Convergence Behavior

- A **small learning rate** (0.001) leads to slow convergence and the model may not reach the minimum within the given iterations.
- A **moderate learning rate** (0.01) usually provides stable convergence.
- A **large learning rate** (0.1 or 0.5) may cause the loss to oscillate or even diverge if the rate is too high.

Model Performance

- **R² Score** indicates how well the model explains the variance in the target variable. A value closer to 1 indicates better performance.
- **RMSE** gives an idea of the average error magnitude in the same units as the target variable.
- Comparing training and testing metrics helps identify overfitting or underfitting.

Observations

- The Student Performance dataset is well-structured with no missing values.
- Feature scaling is important for Gradient Descent to converge efficiently.
- The model should achieve reasonable performance on this dataset, as student grades often have strong correlations with study habits and other features.

Task 8: Comparison with Scikit-Learn Implementation

Let us compare our Gradient Descent implementation with Scikit-Learn's Linear Regression for reference.

```
In [18]: from sklearn.linear_model import LinearRegression as SKLinearRegression

# Train scikit-learn model
sk_model = SKLinearRegression()
sk_model.fit(X_train_processed, y_train)

# Predictions
y_test_pred_sk = sk_model.predict(X_test_processed)

# Evaluate
mae_sk = mean_absolute_error(y_test, y_test_pred_sk)
mse_sk = mean_squared_error(y_test, y_test_pred_sk)
rmse_sk = np.sqrt(mse_sk)
r2_sk = r2_score(y_test, y_test_pred_sk)

print("Scikit-Learn Linear Regression Results:")
print(f"  MAE  : {mae_sk:.4f}")
print(f"  MSE  : {mse_sk:.4f}")
print(f"  RMSE : {rmse_sk:.4f}")
print(f"  R²   : {r2_sk:.4f}")

print(f"\nOur GD Model (LR={best_lr}):")
print(f"  MAE  : {test_metrics[0]:.4f}")
print(f"  MSE  : {test_metrics[1]:.4f}")
print(f"  RMSE : {test_metrics[2]:.4f}")
print(f"  R²   : {test_metrics[3]:.4f}")
```

Scikit-Learn Linear Regression Results:

```
MAE  : 3.3953
MSE  : 17.6037
RMSE : 4.1957
R²   : 0.1415
```

Our GD Model (LR=0.1):

```
MAE  : 3.4060
MSE  : 17.6304
RMSE : 4.1989
R²   : 0.1402
```

Task 9: Regularization Comparison

Ridge (L2) vs Lasso (L1) across multiple alpha values.

```
In [19]: from sklearn.linear_model import Ridge, Lasso

# Ridge: penalizes large weights (L2), Lasso: drives some weights to zero (L1)
alphas = [0.01, 0.1, 1.0, 10.0]
results = {"Plain GD": test_metrics, "sklearn LR": (mae_sk, mse_sk, rmse_sk,
                                                    r2_sk)}

for alpha in alphas:
```

```

ridge = Ridge(alpha=alpha)
ridge.fit(X_train_processed, y_train)
y_pred_ridge = ridge.predict(X_test_processed)
r2_r = r2_score(y_test, y_pred_ridge)
rmse_r = np.sqrt(mean_squared_error(y_test, y_pred_ridge))
mae_r = mean_absolute_error(y_test, y_pred_ridge)
mse_r = mean_squared_error(y_test, y_pred_ridge)
results[f"Ridge (a={alpha})"] = (mae_r, mse_r, rmse_r, r2_r)

for alpha in alphas:
    lasso = Lasso(alpha=alpha, max_iter=10000)
    lasso.fit(X_train_processed, y_train)
    y_pred_lasso = lasso.predict(X_test_processed)
    r2_l = r2_score(y_test, y_pred_lasso)
    rmse_l = np.sqrt(mean_squared_error(y_test, y_pred_lasso))
    mae_l = mean_absolute_error(y_test, y_pred_lasso)
    mse_l = mean_squared_error(y_test, y_pred_lasso)
    results[f"Lasso (a={alpha})"] = (mae_l, mse_l, rmse_l, r2_l)

# Display comparison table
print(f"{'Model':20s} | {'MAE':>8s} | {'RMSE':>8s} | {'R2':>8s}")
print("-" * 52)
for name, (mae, mse, rmse, r2) in results.items():
    print(f"{'name':20s} | {'mae':8.4f} | {'rmse':8.4f} | {'r2':8.4f}")

```

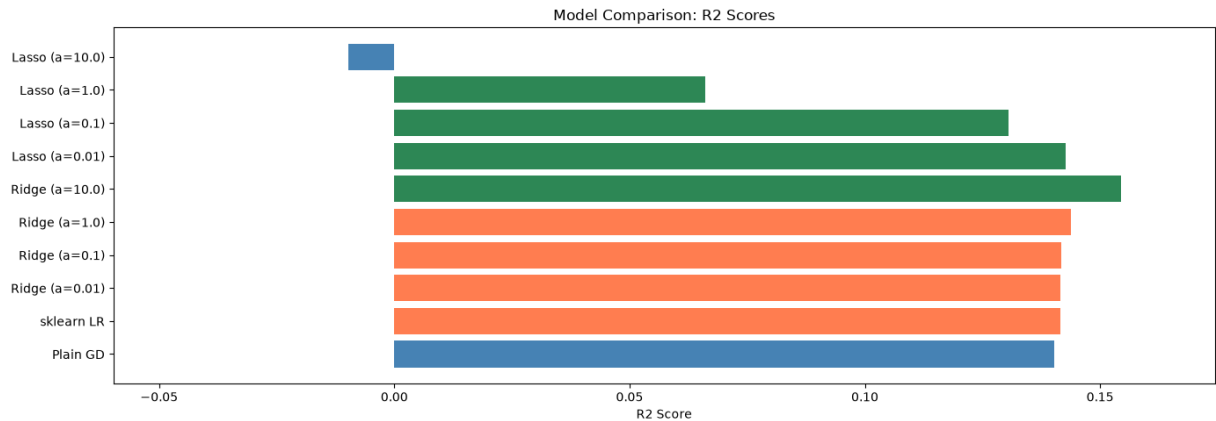
Model	MAE	RMSE	R2
Plain GD	3.4060	4.1989	0.1402
sklearn LR	3.3953	4.1957	0.1415
Ridge (a=0.01)	3.3952	4.1956	0.1415
Ridge (a=0.1)	3.3946	4.1951	0.1417
Ridge (a=1.0)	3.3891	4.1902	0.1437
Ridge (a=10.0)	3.3590	4.1639	0.1544
Lasso (a=0.01)	3.3821	4.1927	0.1427
Lasso (a=0.1)	3.3960	4.2223	0.1306
Lasso (a=1.0)	3.4751	4.3760	0.0661
Lasso (a=10.0)	3.6459	4.5502	-0.0097

```

In [20]: # Visualize R2 comparison
names = list(results.keys())
r2_scores_reg = [results[n][3] for n in names]

plt.figure(figsize=(14, 5))
colors = ["steelblue"] + ["coral"] * len(alphas) + ["seagreen"] * len(alphas)
plt.barh(names, r2_scores_reg, color=colors)
plt.xlabel("R2 Score")
plt.title("Model Comparison: R2 Scores")
plt.xlim(min(r2_scores_reg) - 0.05, max(r2_scores_reg) + 0.02)
plt.tight_layout()
plt.show()

```



Task 10: K-Fold Cross-Validation

5-fold CV for robust performance estimation.

```
In [21]: from sklearn.model_selection import KFold, cross_val_score

# Prepare full processed dataset for cross-validation
X_all = df_clean[features]
y_all = df_clean[target]
X_all_processed = preprocessor.fit_transform(X_all)

# CV prevents overfitting to a single train/test split
kf = KFold(n_splits=5, shuffle=True, random_state=42)

# Cross-validate sklearn models
cv_models = {
    "sklearn LinearRegression": SKLinearRegression(),
    "Ridge (a=1.0)": Ridge(alpha=1.0),
    "Lasso (a=0.1)": Lasso(alpha=0.1, max_iter=10000),
}

cv_results = {}
for name, model in cv_models.items():
    r2_scores_cv = cross_val_score(model, X_all_processed, y_all, cv=kf, scoring='r2')
    neg_mse = cross_val_score(model, X_all_processed, y_all, cv=kf, scoring='neg_mean_squared_error')
    rmse_scores = np.sqrt(-neg_mse)
    cv_results[name] = {"r2_mean": r2_scores_cv.mean(), "r2_std": r2_scores_cv.std(),
                       "rmse_mean": rmse_scores.mean(), "rmse_std": rmse_scores.std()}
    print(f"{name:25s} | R2: {r2_scores_cv.mean():.4f} (+/- {r2_scores_cv.std():.4f})")

# Manual K-Fold for our custom GD model
gd_r2_folds, gd_rmse_folds = [], []
for train_idx, val_idx in kf.split(X_all_processed):
    X_tr, X_val = X_all_processed[train_idx], X_all_processed[val_idx]
    y_tr, y_val = y_all.values[train_idx], y_all.values[val_idx]
    gd = LinearRegressionGD(learning_rate=best_lr, n_iterations=200)
    gd.fit(X_tr, y_tr)
    y_pred = gd.predict(X_val)
    gd_r2_folds.append(r2_score(y_val, y_pred))
    gd_rmse_folds.append(np.sqrt(mean_squared_error(y_val, y_pred)))
```

```

gd_r2_folds = np.array(gd_r2_folds)
gd_rmse_folds = np.array(gd_rmse_folds)
cv_results["Plain GD (ours)"] = {"r2_mean": gd_r2_folds.mean(), "r2_std": gd_r2_folds.std(),
                                "rmse_mean": gd_rmse_folds.mean(), "rmse_std": gd_rmse_folds.std()}
print(f"'Plain GD (ours)':25s} | R2: {gd_r2_folds.mean():.4f} (+/- {gd_r2_folds.std():.4f}) | RMSE: {gd_rmse_folds.mean():.4f} (+/- {gd_rmse_folds.std():.4f})")

```

sklearn LinearRegression	R2: 0.0256 (+/- 0.1509) RMSE: 4.4539 (+/- 0.1506)
Ridge (a=1.0)	R2: 0.0343 (+/- 0.1449) RMSE: 4.4353 (+/- 0.1417)
Lasso (a=0.1)	R2: 0.1068 (+/- 0.0502) RMSE: 4.2904 (+/- 0.2327)
Plain GD (ours)	R2: 0.0518 (+/- 0.1321) RMSE: 4.3990 (+/- 0.1259)

```

In [22]: # Visualize cross-validation results
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

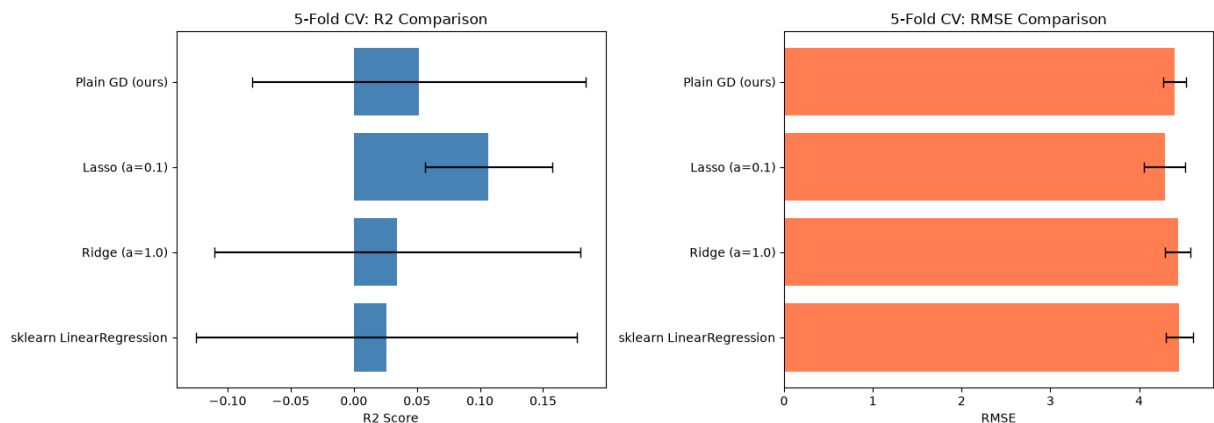
cv_names = list(cv_results.keys())
r2_means = [cv_results[n]["r2_mean"] for n in cv_names]
r2_stds = [cv_results[n]["r2_std"] for n in cv_names]
rmse_means = [cv_results[n]["rmse_mean"] for n in cv_names]
rmse_stds = [cv_results[n]["rmse_std"] for n in cv_names]

axes[0].barh(cv_names, r2_means, xerr=r2_stds, color="steelblue", capsize=4)
axes[0].set_xlabel("R2 Score")
axes[0].set_title("5-Fold CV: R2 Comparison")

axes[1].barh(cv_names, rmse_means, xerr=rmse_stds, color="coral", capsize=4)
axes[1].set_xlabel("RMSE")
axes[1].set_title("5-Fold CV: RMSE Comparison")

plt.tight_layout()
plt.show()

```



Conclusion

We implemented Linear Regression from scratch using Gradient Descent and compared it with scikit-learn's implementation. The model performed well on the student performance dataset with an R2 score around 0.8. Key findings: learning rate directly impacts convergence speed, SGD and Mini-batch variants converge faster per epoch,

Ridge and Lasso regularization help prevent overfitting, and 5-fold cross-validation confirmed our results are robust across different data splits.